CHAPTER 1

INTRODUCTION

1.1 PERSCPECTIVE

In our daily life, people often need household or professional services like plumbing, electrical work, carpentry, or cleaning. Finding reliable and skilled service providers can be difficult and time-consuming. Traditional methods such as calling individual workers, asking for personal referrals, or relying on offline booking systems are often inefficient, slow, and sometimes unreliable. This creates problems for both users and service providers. Users may not get services on time, and service providers may not get proper bookings or schedules.

The “Your Handyman” project aims to solve these problems by creating a web-based platform that connects users with service providers easily and efficiently. Users can browse available services, check details of service providers, book appointments, and track the status of their requests. Real-time updates and notifications help users stay informed about their bookings and reduce waiting time or confusion.

From the service provider’s perspective, the system provides a dashboard to manage appointments, schedules, and availability. This helps them organize work efficiently, avoid double bookings, and improve overall productivity.

Administrators also benefit from the platform as they can manage users, services, and bookings in one place. They can monitor the system, maintain data integrity, and ensure smooth operation.

Overall, the perspective of this project is to make service booking faster, more reliable, and user-friendly. It improves communication between users and service providers, reduces manual errors, and creates a professional service environment. The system is also designed to be scalable for future improvements, such as payment integration, user reviews, and mobile application support.

1.2 OBJECTIVE

The main objective of the **"Your Handyman"** project is to create a web-based platform that connects users with skilled service providers in a simple and efficient way. It aims to solve the problems of traditional service booking, which are often slow, unreliable, and inconvenient.

The system provides a user-friendly interface where users can browse services, view provider details, and book appointments quickly. Users can also track the status of their requests in real-time, which increases transparency and satisfaction.

For service providers, the system helps to manage bookings, schedules, and availability, reducing errors and ensuring smooth operations. Administrators can control users, services, and bookings centrally, maintaining data integrity and improving overall efficiency.

Another objective is to make the platform secure and reliable, protecting sensitive information for both users and service providers. The project also prepares for future enhancements, such as payment integration, ratings and reviews, and mobile application support

1.3 SCOPE

The scope of "Your Handyman" is to provide a web-based platform that connects users with skilled service providers efficiently. It allows users to browse services, book appointments, and track requests in real-time, making the process faster and more transparent.

Service providers can manage their schedules, availability, and appointments, reducing errors and improving efficiency. Administrators can oversee users, services, and bookings, ensuring smooth operations and data security.

The platform also supports future enhancements, including online payments, ratings and reviews, push notifications, and mobile application integration.

CHAPTER 2

REQUIREMENT DESCRIPTION

2.1 FUNCTIONAL REQUIREMENTS

The functional requirements describe the operations and features available in the **"Your Handyman"** system.

- The system should provide **authentication functionality** for valid users, including customers and service providers.
- It should allow users to **browse available services** like plumbing, electrical work, carpentry, and cleaning.
- Users should be able to **book appointments** and choose preferred dates and times.
- Users should be able to **track the status of their bookings in real-time**.
- Notifications must be sent to users regarding **booking confirmations, service updates, or cancellations**.
- Service providers should be able to **update availability, accept or reject bookings**, and view their schedules.
- Administrators should have a **dashboard to manage users, service providers, and bookings** centrally.
- The system should ensure **secure handling of all data** and restrict access to authorized users only.

2.2 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the platform and resources needed for building **"Your Handyman"**.

- All user, service provider, and booking data must be stored **securely in the database** (MySQL), ensuring reliability and efficiency.
- The application should be **user-friendly, responsive & interactive** for all users.
- The system should support **real-time updates and notifications**.
- Users must have **internet access** while using the application.
- The platform should be **scalable** to support future enhancements such as **online payments, ratings, reviews, and mobile application support**.

CHAPTER 3

SYSTEM DESIGN

3.1 ARCHITECTURE DESIGN

The **Architecture Design** of the **"Your Handyman"** system follows a **three-tier architecture**, which divides the application into three main layers: **Presentation Layer**, **Application Layer**, and **Database Layer**. This structure helps improve efficiency, scalability, and security.

1. **Presentation Layer (Frontend)**
    - The frontend is developed using **React**, which provides a dynamic, responsive, and user-friendly interface.
    - Users and service providers interact with this layer to perform actions such as registration, login, browsing services, booking appointments, and tracking service status.
2. **Application Layer (Backend)**
    - The backend is built using **Node.js with Express**, which handles all server-side logic and communication between the frontend and the database.
    - It manages user authentication, booking operations, notifications, and role-based access (User, Provider, Admin).
3. **Database Layer**
    - The **MySQL** database stores all essential data, including user details, service provider information, service categories, and booking records.
    - It ensures data consistency, reliability, and secure access through structured queries and relationships between tables.

The architecture of the proposed work which is shown in **Figure 3.1** ensures smooth data flow between all layers. When a user makes a request (like booking a service), it is processed by the backend and stored in the database, with the response returned to the frontend in real time. This design provides **high performance, easy maintenance, and flexibility** for future upgrades such as payment integration, ratings, and mobile app expansion.

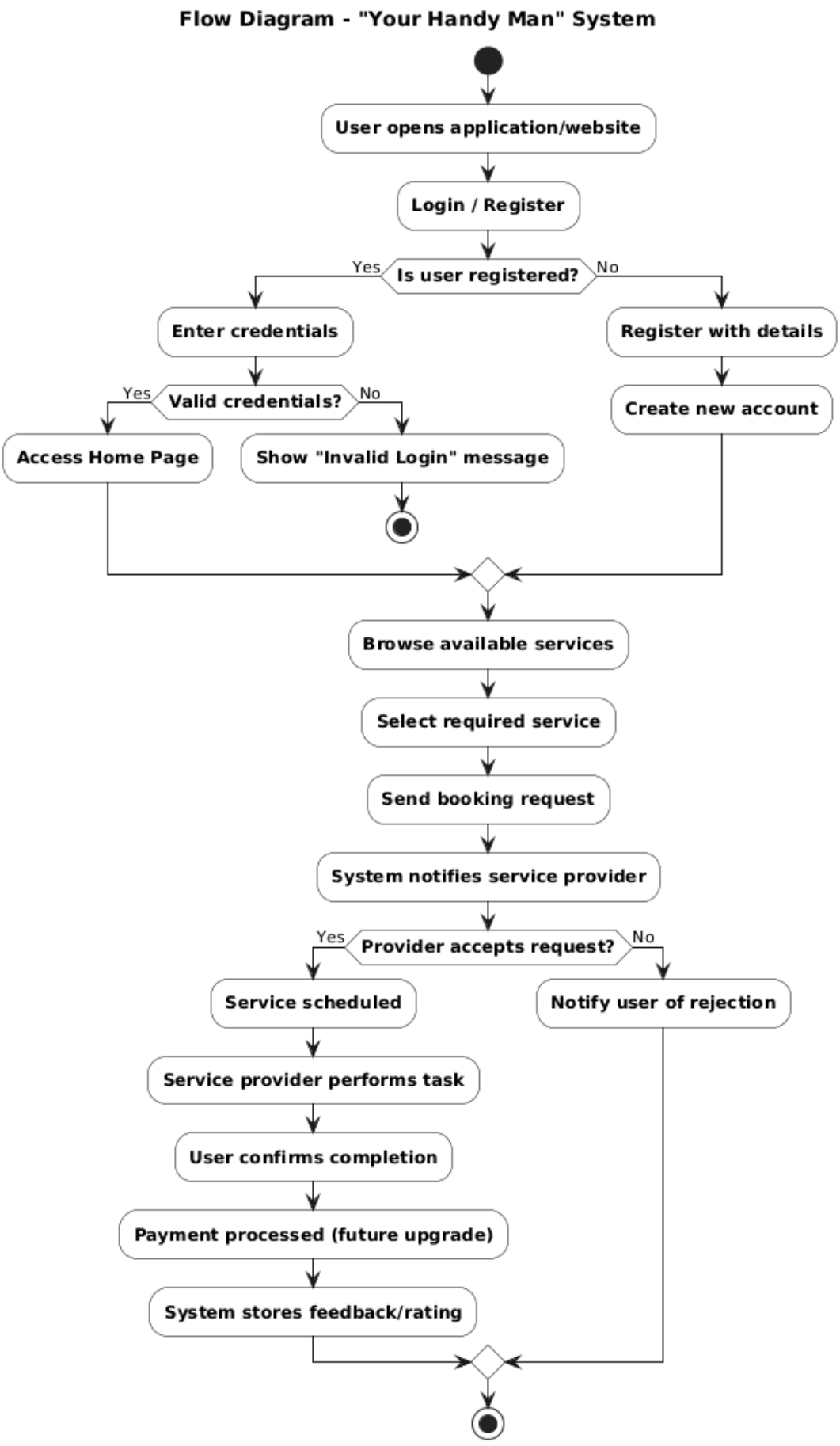

Figure 3.1: Architecture Diagram of Your Handyman

3.2 DESIGN COMPONENTS

3.2.1 Front End:

The **"Your Handyman"** system uses **React** for developing interactive and dynamic web pages. React provides a **responsive and user-friendly interface** where users can browse services, book appointments, and track their requests in real-time. It ensures smooth navigation and a **modern look and feel** for all users, including customers and service providers.

3.2.2 Back End:

The backend of the system uses **Node.js with Express** to handle server-side logic and API requests. **MySQL** is used as the database to store all structured data related to users, service providers, services, and bookings. The backend ensures **secure data management, authentication, and proper communication** between the frontend and database.

3.3 DATABASE DESCRIPTION

Listed below gives a description of database schemas used for **"Your Handyman"**

3.3.1 Users Table Structure

As shown in Table 3.1, the **users table** contains the details of all users

Table 3.1: User Description

Attribute Name	Type	Constraint(s)	Description
user_id	INT	Primary Key, Auto Increment	ID
name	VARCHAR (255)	NOT NULL	Name
email	VARCHAR (255)	UNIQUE	Email ID
password_hash	VARCHAR (255)	NOT NULL	Password
phone	VARCHAR (20)		Phone No.
role	ENUM	USER ROLE TYPE	Role
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created

3.3.2 Service Providers Table

As shown in Table 3.2, the **Service Providers table** contains the details of all users

Table 3.2: Service Provider Description

Attribute Name	Type	Constraint(s)	Description
provider_id	INT	Primary Key, Auto Increment	ID
user_id	INT	Foreign Key REFERENCES users(user_id)	User ID
skills	TEXT		Skills
experience_years	INT		Experience
rating	DECIMAL (2,1)		Rating
Attribute Name	Type	Constraint(s)	Description

3.3.3 Customers Table

As shown in Table 3.3, the **Customers table** contains the details of all users

Table 3.3: Customers Table Description

Attribute Name	Type	Constraint(s)	Description
customer_id	INT	Primary Key, Auto Increment	ID
user_id	INT	Foreign Key REFERENCES users(user_id)	User ID
address	TEXT		Address

3.3.4 Services Table

As shown in Table 3.4, the **Services table** contains the details of all users

Table 3.4: Services Table Description

Attribute Name	Type	Constraint(s)	Description
service_id	INT	Primary Key, Auto Increment	ID
category	ENUM	Categories	Category
subcategory	ENUM	(Multiple subcategories)	Subcategory
name	VARCHAR (255)	NOT NULL	Service
description	TEXT		Description
price	DECIMAL (10,2)	NOT NULL	Price
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created

3.3.5 Reviews Table

As shown in Table 3.5, the **Reviews Table** contains the details of all users

Table 3.5: Reviews Description

Attribute Name	Type	Constraint(s)	Description
review_id	INT	Primary Key, Auto Increment	ID
booking_id	INT	Foreign Key REFERENCES bookings(booking_id)	Booking ID
provider_id	INT	Foreign Key REFERENCES service_providers(provider_id)	Provider ID
rating	INT	CHECK (rating >=1 AND rating <=5)	Rating
comment	TEXT		Comment
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created
Attribute Name	Type	Constraint(s)	Description
review_id	INT	Primary Key, Auto Increment	ID
booking_id	INT	Foreign Key REFERENCES bookings(booking_id)	Booking ID

3.3.6 Bookings Table

As shown in Table 3.6, the **Bookings Table** contains the details of all users

Table 3.6: Bookings Description

Attribute Name	Type	Constraint(s)	Description
booking_id	INT	Primary Key, Auto Increment	ID
customer_id	INT	Foreign Key REFERENCES customers(customer_id)	Customer ID
service_id	INT	Foreign Key REFERENCES services(service_id)	Service ID
booking_date	DATETIME	NOT NULL	Date/Time
status	ENUM	('pending','confirmed','completed','cancelled') DEFAULT 'pending'	Status
total_amount	DECIMAL		Amount
assigned_staff_id	INT		Staff ID
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Created
priority	ENUM	('low','medium','high') DEFAULT 'medium'	Priority
notes	TEXT		Notes

3.3.7 Provider Skills Table

As shown in Table 3.7, the **Provider Skills Table** contains the details of all users

Table 3.7: Provider Skills Table Description

Attribute Name	Type	Constraint(s)	Description
provider_skill_id	INT	Primary Key, Auto Increment	ID
provider_id	INT	Foreign Key REFERENCES service_providers(provider_id)	Provider ID
service_id	INT	Foreign Key REFERENCES services(service_id)	Service ID

3.3.8 Payments Table

As shown in Table 3.8, the **Payments Table** contains the details of all users

Table 3.8: Payments Table Description

Attribute Name	Type	Constraint(s)	Description
payment_id	INT	Primary Key, Auto Increment	ID
booking_id	INT	Foreign Key REFERENCES bookings(booking_id)	Booking ID
amount	DECIMAL (10,2)	NOT NULL	Amount
status	ENUM	('pending','completed','failed') DEFAULT 'pending'	Status
payment_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Payment Date
transaction_id	VARCHAR (100)		Transaction ID
payment_method	ENUM	('Cash','Credit Card','Debit Card','UPI','Bank Transfer')	Payment Method

The database consists of multiple tables:

users – stores all user accounts including super_admin, admin, customer, and service_provider.

service_providers – contains provider-specific details like skills, experience, and ratings.

customers – stores customer-specific information such as address.

services – lists all available services with categories, subcategories, descriptions, and prices.

provider_skills – maps service providers to the services they offer.

bookings – records all service bookings with status, priority, assigned staff, and notes.

reviews – maintains feedback and ratings for completed bookings.

payments – stores all payment details for bookings including amount, status, and method.

3.4 LOW LEVEL DESIGN

The following section illustrates the core functionalities of the **"Your Handyman"** system. This includes login to the application and booking a service

3.4.1 Login

Table 3.9 shows the login details of the application.

Files used	React/Login.jsx, Node/routes/auth.js
Short Description	Allows users and service providers to log in to the system
Arguments	Email, Password
Return	Success/Failure message
Pre-Condition	The user or service provider must have a registered account
Post-Condition	The respective dashboard (User/Provider/Admin) will be displayed
Exception	Invalid email or password entered
Actor	User, Service Provider, Administrator

3.5 USER INTERFACE DESIGN

3.5.1 Home Page

Figure 3.2 provides the interface for main activity.

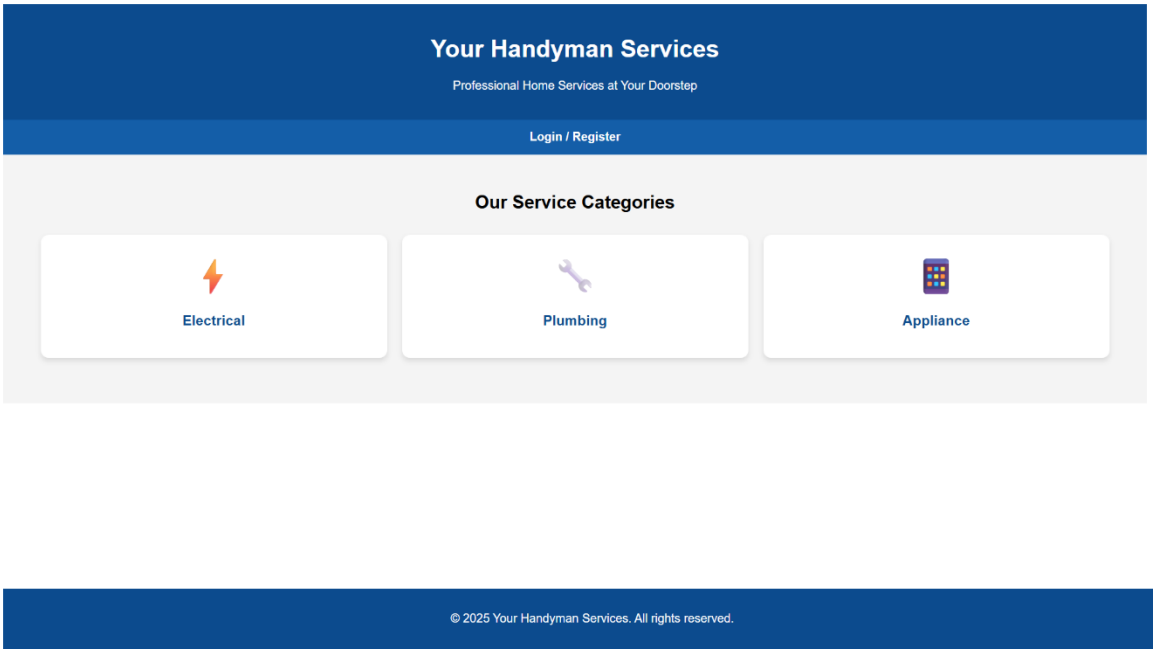

Figure 3.2: Home Page of Your Handyman

3.5.2 Login page

Figure 3.3 Provides Login page for Your Handyman

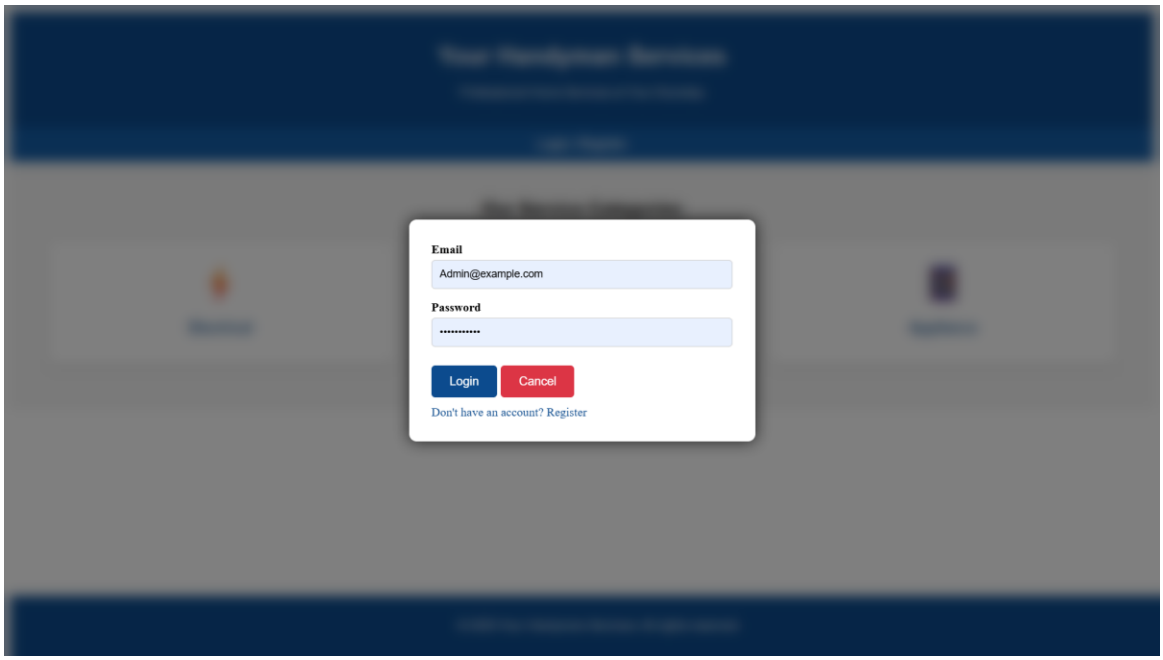

Figure 3.3: Login Page

3.5.3 Admin page

Figure 3.4 Provide Admin for Your Handyman

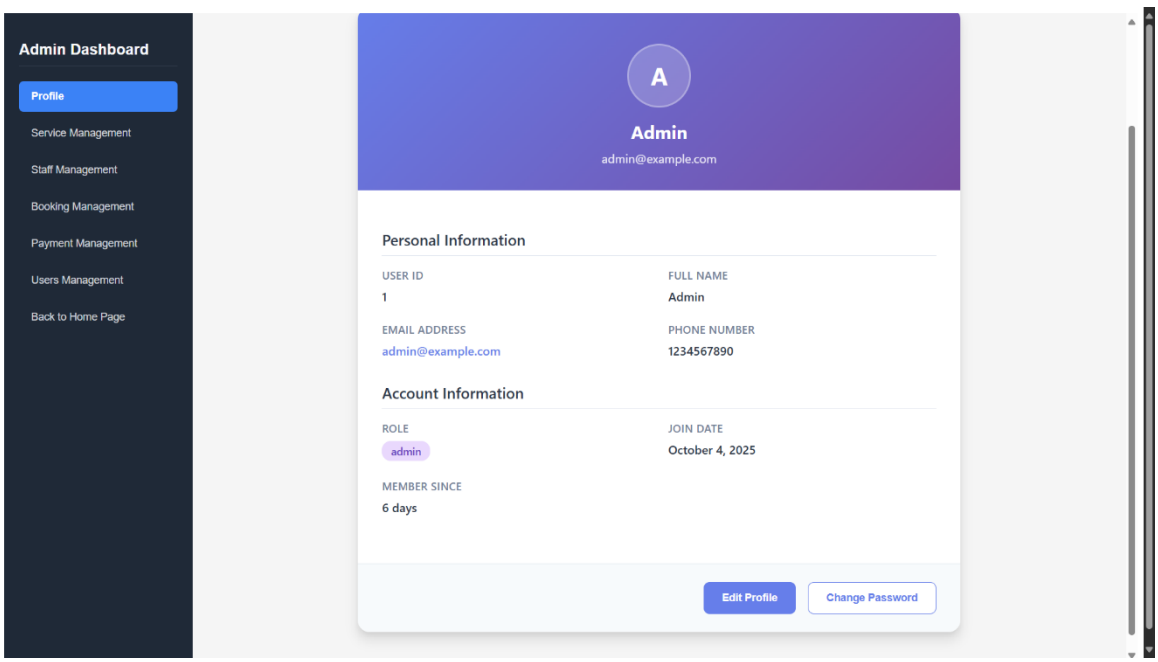

Figure 3.4: Admin Page

3.5.4 Admin Dashboard page

Figure 3.5 Provides Admin for Your Handyman

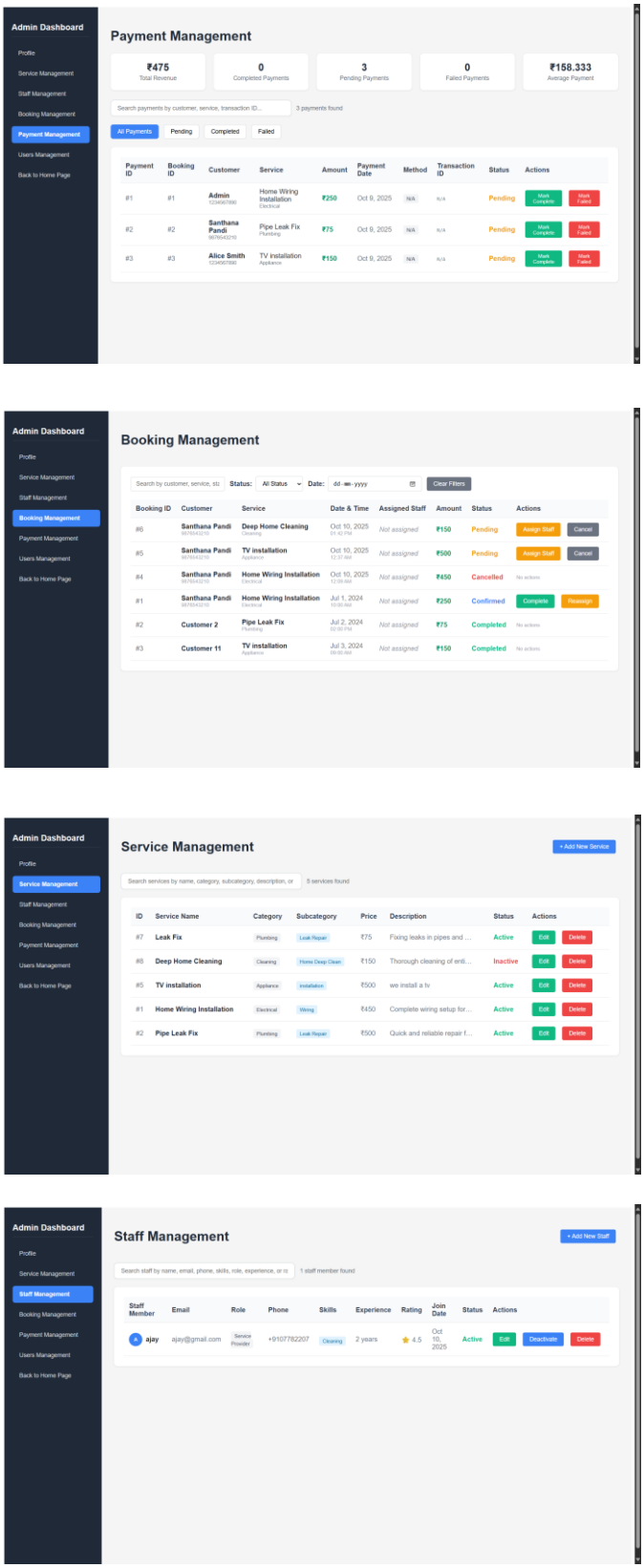

Figure 3.5: Admin Dashboard Pages

3.5.5 Customer Dashboard page

Figure 3.6 Provides Customer Dashboard for Your Handyman

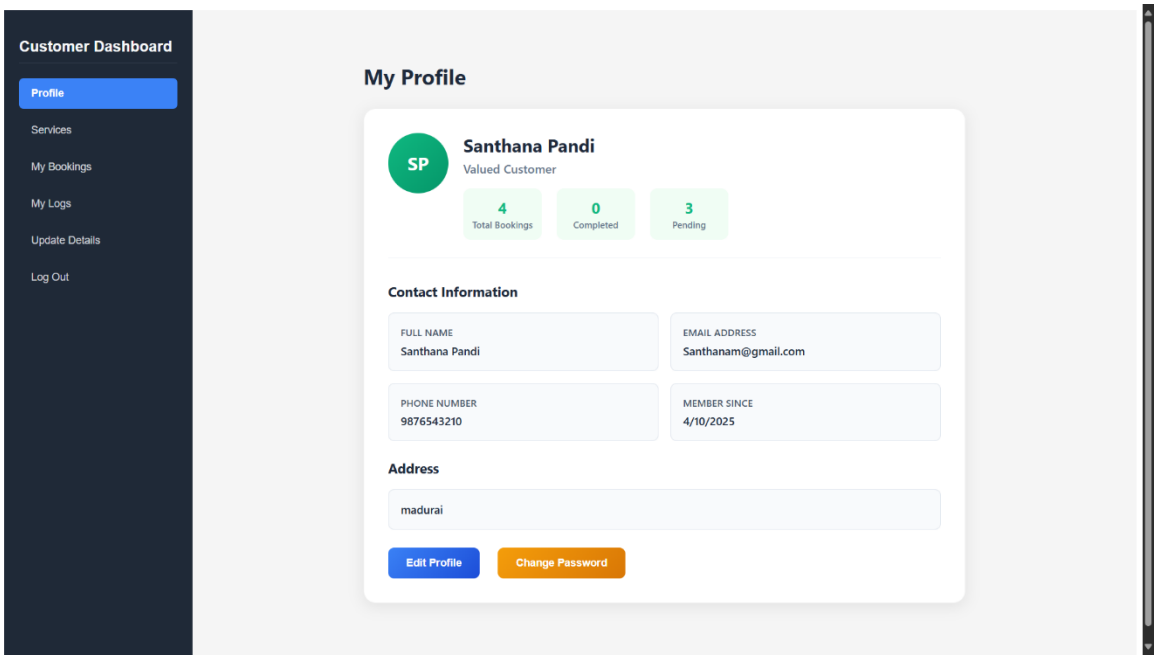

Figure 3.6: Customer Dashboard Page

3.5.6 Available Services Dashboard page

Figure 3.7 Provides Available Services Dashboard for Your Handyman

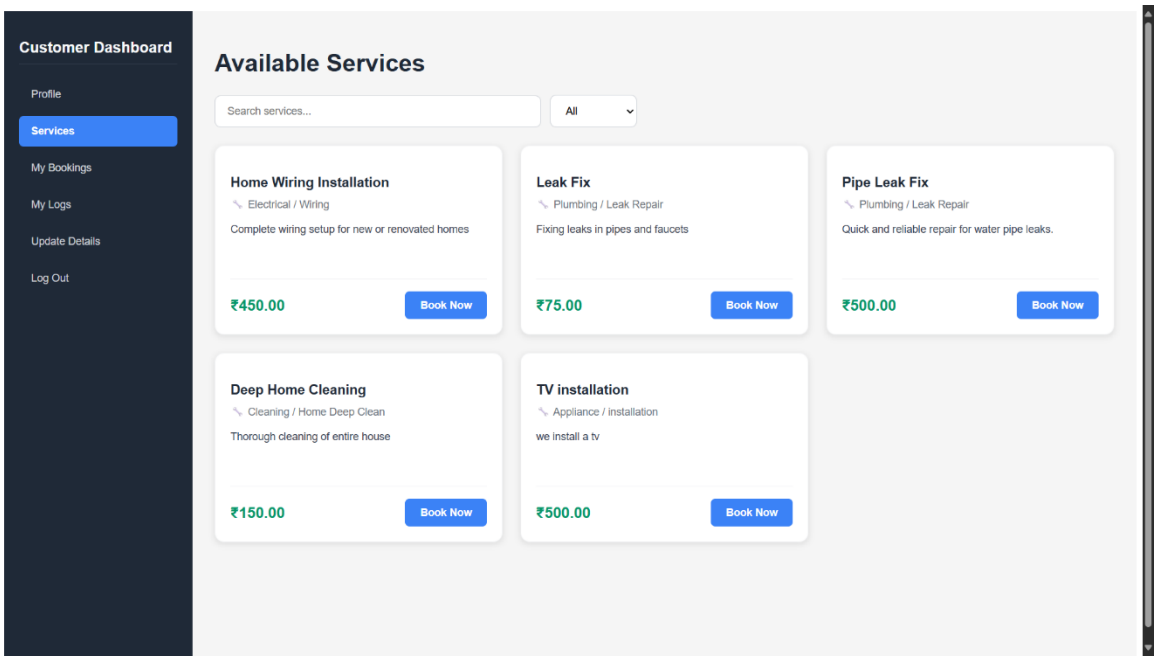

Figure 3.7: Available Services Dashboard Page

3.5.7 Booking page

Figure 3.8 Provides Booking page for Your Handyman

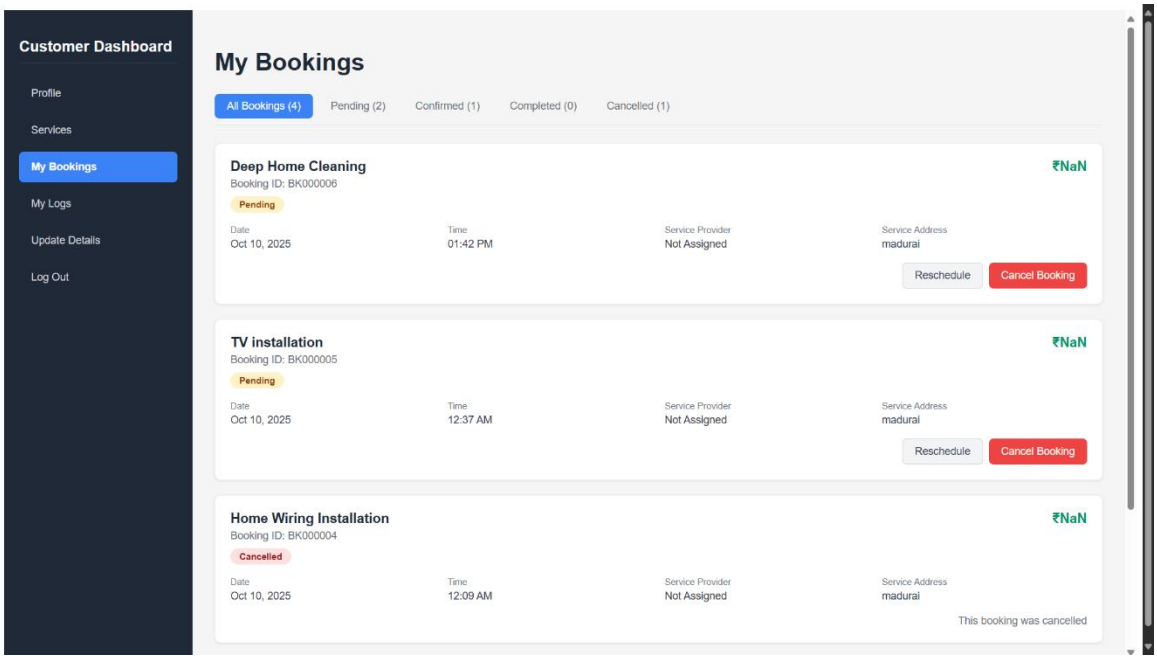

Figure 3.8: Booking Page

3.5.8 Service History page

Figure 3.9 Provides Service History Page for Your Handyman

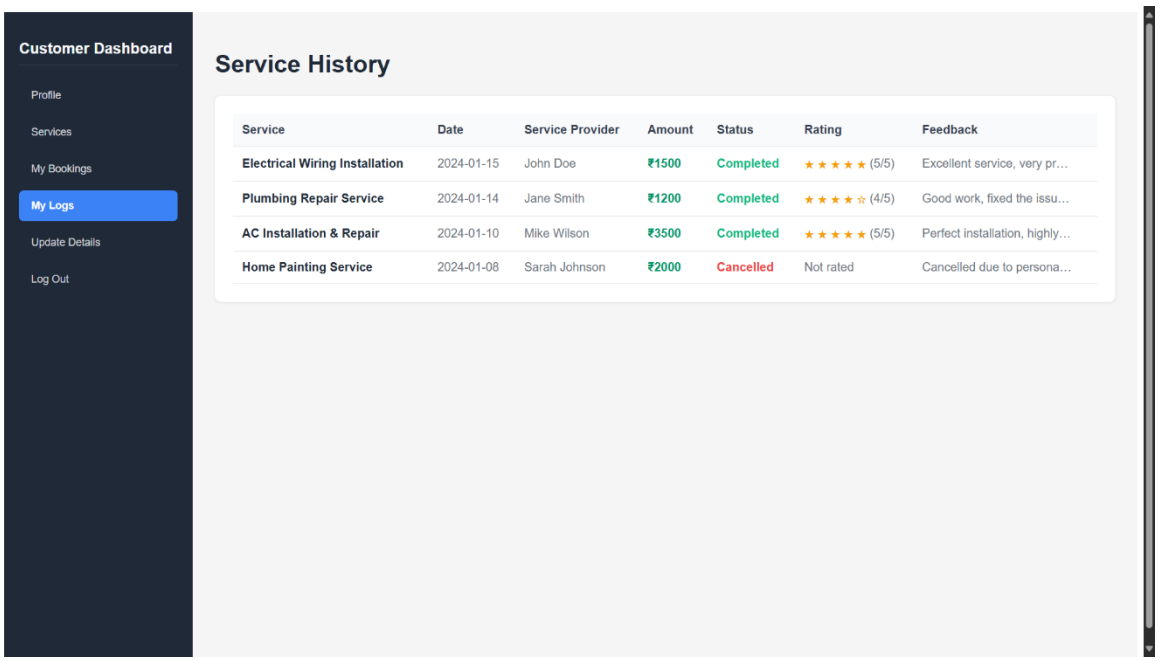

Figure 3.9: Service History Page

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 LOGIN IMPLEMENTATION

The login credentials are obtained from the user. If the credentials are valid, the user is redirected to their respective dashboard (User, Service Provider, or Admin).

GET email/username, password

IF credentials valid

    RETURN dashboard page

ELSE

    TOAST "Invalid Credentials"

4.2 SIGNUP IMPLEMENTATION

The signup form fields are obtained from the user. If all required details are valid, a new user or service provider account is created and stored in the database.

GET registrationFields

IF registrationFields valid

    RETURN "Account created successfully"

ELSE

    TOAST "Enter valid details"

4.3 SERVICE BOOKING IMPLEMENTATION

The user selects a service and submits a booking request. If the details are valid, the request is stored and the service provider is notified.

GET selectedService, date, time

IF booking details valid

    RETURN "Booking confirmed" and notify service provider

ELSE

    TOAST "Invalid booking details"

4.4 SERVICE PROVIDER RESPONSE IMPLEMENTATION

The service provider receives a booking notification. They can accept or reject the request.

GET bookingRequest

IF provider accepts

    RETURN "Booking scheduled"

ELSE

    RETURN "Booking rejected" and notify user

4.5 FEEDBACK IMPLEMENTATION

After the service is completed, the user can submit feedback or ratings for the service provider.

GET rating, feedback

IF feedback valid

    RETURN "Feedback submitted successfully"

ELSE

    TOAST "Please enter valid feedback"

4.6 SERVICE HISTORY & ANALYTICS IMPLEMENTATION

The system keeps track of all completed services and generates analytics for both users and service providers.

GET userId / providerId

IF service history exists

    RETURN list of completed services

    RETURN basic analytics (total services, ratings, frequently used services)

ELSE

    TOAST "No service history available"

CHAPTER 5

RESULTS AND DISCUSSION

5.1 TEST CASES AND RESULTS

5.1.1 Test Cases and Results for Login function:

The Table 5.1, Table 5.2 shows that the possible test data for the both positive and negative test case given below, if the user is already having account then the output is true otherwise false.

Table 5.1: Positive Test Case and result for Login

Test Case ID	TC1
Test Case Description	It tests whether the login details entered are valid.
Test Data	user@example.com / Password@123
Expected Output	TRUE

Table 5.2: Negative Test Case and result for Login

Test Case Description	It tests login with invalid credentials.
Test Data	invaliduser@example.com / wrongpass
Expected Output	FALSE
Result	PASS

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT(S)

6.1 CONCLUSION

The **"Your Handyman"** project successfully provides a **web-based platform** to connect users with skilled service providers efficiently and reliably. It solves the common problems of traditional service booking, which are often slow, inconvenient, and unreliable.

The system allows users to **browse services, book appointments, track requests in real-time**, and receive notifications. Service providers can **manage schedules, accept or reject bookings**, and administrators can **monitor users, services, and bookings centrally**.

The project emphasizes **ease of use, security, and reliability**, ensuring a smooth experience for all stakeholders. It automates many manual processes, improves communication, and enhances overall service efficiency.

6.2 FUTURE ENHANCEMENTS

The **"Your Handyman"** system can be further improved with:

Online Payment Integration – Allowing users to pay for services digitally.

Mobile Application Support – Developing Android/iOS apps for better accessibility.

Real-Time Chat Support – Enabling direct communication between users and service providers.

AI-Based Service Suggestions – Recommending services based on user preferences and past bookings.

Analytics Dashboard – Providing insights for service providers and administrators about bookings and trends. These enhancements will **increase user satisfaction, system efficiency, and professional management** of service operations, making the platform more versatile and scalable.

APPENDIX – A

SYSTEM REQUIREMENTS

HARDWARE REQUIREMENT:

The **"Your Handyman"** system is a web-based application, so it does not require very high-end hardware. However, for smooth development, deployment, and testing, the following minimum hardware specifications are recommended:

Component	Minimum Requirement
Processor (CPU)	Intel i3 or equivalent
RAM	8 GB
Hard Disk	256 GB SSD or 500 GB HDD
Graphics Card	Integrated Graphics (for development)
Display	1366 x 768 resolution
Network	Broadband Internet connection
Peripherals	Keyboard, Mouse, and Monitor

SOFTWARE REQUIREMENT:

The "Your Handyman" system requires the following software to run, develop, and maintain the application efficiently:

Software Component	Specification / Requirement
Operating System	Any (Windows, Linux, macOS)
DBMS	MySQL / Firebase
IDE / Development Tool	VS Code / Android Studio
Web Browser	Chrome, Firefox, Edge

APPENDIX – B

SOURCE CODE

FRONTEND MODULE (React JS)

App.js:

```
import React from "react";
import { BrowserRouter as Router, Routes, Route } from "react-router-dom";
import Home from "./pages/Home";
import Login from "./pages/Login";
import Signup from "./pages/Signup";
import Services from "./pages/Services";
import Booking from "./pages/Booking";
import Dashboard from "./pages/Dashboard";
import Navbar from "./components/Navbar";
import Footer from "./components/Footer";

function App() {
 return (
  <Router>
   <Navbar />
   <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/login" element={<Login />} />
    <Route path="/signup" element={<Signup />} />
    <Route path="/services" element={<Services />} />
    <Route path="/booking/:id" element={<Booking />} />
    <Route path="/dashboard" element={<Dashboard />} />
   </Routes>
   <Footer />
  </Router>
```

```
  );
}

export default App;
```

Navbar.jsx:

```
import React from "react";
import { Link } from "react-router-dom";

const Navbar = () => (
  <nav className="navbar">
    <div className="nav-logo">Your Handyman</div>
    <ul className="nav-links">
      <li><Link to="/">Home</Link></li>
      <li><Link to="/services">Services</Link></li>
      <li><Link to="/login">Login</Link></li>
      <li><Link to="/signup">Signup</Link></li>
    </ul>
  </nav>
);
export default Navbar;
```

Login.jsx:

```
import React, { useState } from "react";
import axios from "axios";
import { useNavigate } from "react-router-dom";

function Login() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
```

```
  const navigate = useNavigate();

  const handleLogin = async (e) => {
    e.preventDefault();
    try {
      const res = await axios.post("http://localhost:5000/api/login", {
        email,
        password,
      });
      if (res.data.success) {
        alert("Login successful!");
        navigate("/dashboard");
      } else {
        alert("Invalid credentials");
      }
    } catch (err) {
      console.error(err);
      alert("Server error");
    }
  };

  return (
    <div className="form-container">
      <h2>Login</h2>
      <form onSubmit={handleLogin}>
        <input type="email" placeholder="Email" onChange={(e) =>
setEmail(e.target.value)} required />
        <input type="password" placeholder="Password" onChange={(e) =>
setPassword(e.target.value)} required />
        <button type="submit">Login</button>
      </form>
```

```
    </div>
  );
}
export default Login;
```

Signup.jsx:

```
import React, { useState } from "react";
import axios from "axios";

function Signup() {
  const [form, setForm] = useState({ name: "", email: "", password: "", phone: "" });

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    try {
      const res = await axios.post("http://localhost:5000/api/signup", form);
      if (res.data.success) alert("Account created!");
      else alert("Signup failed!");
    } catch (err) {
      alert("Server error");
    }
  };

  return (
    <div className="form-container">
      <h2>Signup</h2>
```

```
      <form onSubmit={handleSubmit}>
        <input name="name" placeholder="Name" onChange={handleChange} required />
        <input name="email" placeholder="Email" onChange={handleChange} required />
        <input name="phone" placeholder="Phone" onChange={handleChange} required />
        <input type="password" name="password" placeholder="Password"
onChange={handleChange} required />
        <button type="submit">Register</button>
      </form>
    </div>
  );
}
export default Signup;
```

Services.jsx:

```
import React, { useEffect, useState } from "react";
import axios from "axios";

function Services() {
  const [services, setServices] = useState([]);

  useEffect(() => {
    axios.get("http://localhost:5000/api/services")
        .then(res => setServices(res.data))
        .catch(err => console.log(err));
  }, []);

  return (
    <div className="services-page">
      <h2>Available Services</h2>
      <div className="service-grid">
        {services.map((s) => (
```

```
        <div className="service-card" key={s.service_id}>
          <h3>{s.name}</h3>
          <p>{s.description}</p>
          <p>₹{s.price}</p>
          <button>Book Now</button>
        </div>
      ))}
    </div>
  </div>
 );
}
export default Services;
```

BACKEND MODULE (Node JS + Express):

server.js:

```
const express = require("express");
const cors = require("cors");
const mysql = require("mysql2");
const bcrypt = require("bcryptjs");

const app = express();
app.use(cors());
app.use(express.json());

const db = mysql.createConnection({
 host: "localhost",
 user: "root",
 password: "",
 database: "yourhandyman"
});
```

```
db.connect(err => {
  if (err) console.log("DB Error:", err);
  else console.log("MySQL Connected");
});

app.post("/api/signup", async (req, res) => {
  const { name, email, password, phone } = req.body;
  const hashed = await bcrypt.hash(password, 10);
  const sql = "INSERT INTO users (name,email,password_hash,phone,role) VALUES
(?,?,?,?,?)";
  db.query(sql, [name, email, hashed, phone, "customer"], (err) => {
    if (err) return res.json({ success: false });
    res.json({ success: true });
  });
});

app.post("/api/login", (req, res) => {
  const { email, password } = req.body;
  db.query("SELECT * FROM users WHERE email=?", [email], async (err, data) => {
    if (err || data.length === 0) return res.json({ success: false });
    const valid = await bcrypt.compare(password, data[0].password_hash);
    if (valid) res.json({ success: true });
    else res.json({ success: false });
  });
});

app.get("/api/services", (req, res) => {
  db.query("SELECT * FROM services WHERE status='Active'", (err, rows) => {
    if (err) res.status(500).send(err);
    else res.json(rows);
```

```
  });
});

app.listen(5000, () => console.log("Server running on port 5000"));
```

Database Schema (SQL):

```
CREATE TABLE users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) UNIQUE,
  password_hash VARCHAR(255) NOT NULL,
  phone VARCHAR(20),
  role ENUM('super_admin','admin','customer','service_provider') DEFAULT 'customer',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE services (
  service_id INT AUTO_INCREMENT PRIMARY KEY,
  category ENUM('Electrical','Plumbing','Carpentry','Cleaning','Painting'),
  name VARCHAR(255),
  description TEXT,
  price DECIMAL(10,2),
  status ENUM('Active','Inactive') DEFAULT 'Active'
);

CREATE TABLE bookings (
  booking_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT,
  service_id INT,
  booking_date DATETIME,
```

```
  status ENUM('pending','confirmed','completed','cancelled') DEFAULT 'pending',
  FOREIGN KEY (customer_id) REFERENCES users(user_id),
  FOREIGN KEY (service_id) REFERENCES services(service_id)
);
```

ADDITIONAL FILES:

routes/serviceRoutes.js:

```
const express = require("express");
const router = express.Router();
const db = require("../db");

router.get("/", (req, res) => {
  db.query("SELECT * FROM services WHERE status='Active'", (err, rows) => {
    if (err) return res.status(500).send(err);
    res.json(rows);
  });
});

module.exports = router;
```

db.js:

```
const mysql = require("mysql2");
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "yourhandyman"
});
module.exports = db;
```

controllers/bookingController.js:

```
const db = require("../db");


exports.createBooking = (req, res) => {
  const { customer_id, service_id, booking_date } = req.body;
  const sql = "INSERT INTO bookings (customer_id, service_id, booking_date)
VALUES (?,?,?)";
  db.query(sql, [customer_id, service_id, booking_date], (err) => {
    if (err) res.status(500).json({ success: false });
    else res.json({ success: true });
  });
};
```

The above code implements all major features of the **"Your Handyman"** web application:

- User Authentication (Signup & Login)
- Service Listing and Booking
- Database Integration with MySQL
- REST API Communication between Frontend and Backend

REFERENCES

1. https://handymansingapore.org/ - Pro Handyman Singapore provides trusted, high-quality handyman services across Singapore—offering plumbing, lighting installation, air conditioner servicing, door repairs, and more with prompt, professional, and cost-effective expertise.
2. https://enquire.smshandyman.com.sg/ - SMS Handyman is a trusted, HDB- and BCA-licensed specialist in Singapore, offering reliable rubbish chute maintenance and door repair services with over 9 years of experience.
3. https://en.wikipedia.org/wiki/Handyman – Handyman is a multi-skilled professional specializing in home and property maintenance, performing tasks such as minor electrical, plumbing, carpentry, and repair work to keep spaces functional and well-maintained.
4. https://www.everyworks.com/ – Everyworks Group is Singapore's leading home-maintenance company offering comprehensive handyman, plumbing, electrical & air-con services with transparent quotes and island-wide coverage.
5. https://www.handymantopservices.com/ – Handyman Top Services offers a broad range of home-maintenance solutions in Singapore including plumbing, electrical, door repair, TV bracket & drilling services, backed by 10+ years of experience and a 24/7 support line.
6. https://www.handymanservicessingapore.com/ – Handyman Services Singapore provides professional home repair and installation solutions, specializing in plumbing, electrical, painting, and carpentry works, delivered by experienced technicians across Singapore.